

Assignment 3 Bridge Pattern

Zhannur Bakyt SE-2524

Topic: D

Repository: <https://github.com/zhannurr-png/Bridge-Pattern.git>

Base commit: aa61130 This assignment demonstrates the Bridge Pattern with remote controls and devices.

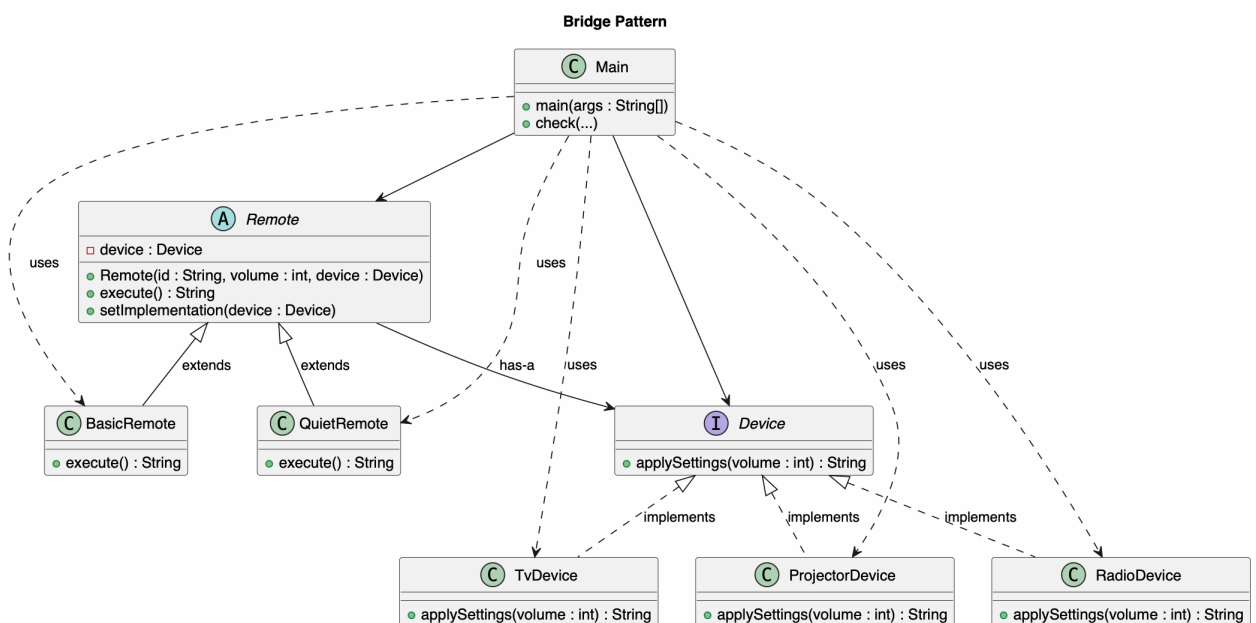
It contains two independent dimensions:

- Abstraction: types of remote control: Remote, BasicRemote, QuietRemote.
- Implementor: types of devices: Device, TvDevice, RadioDevice, ProjectorDevice.

These two dimensions can change independently. For example, a new remote control can be added without changing the existing devices. Similarly, a new device can be added without modifying the remote control classes.

The Bridge Pattern fits this problem because it separates the remote-control abstraction from the device implementation. The Remote class keeps a reference to the device interface instead of depending on one concrete device. This allows any compatible device implementation to be connected to any remote-control abstraction.

1. UML



2. Code excerpts

2.1 Separation of responsibilities

The Remote class is responsible for the remote control, while the Device classes are responsible for the actual devices. The remote does not need to know how a tv, radio, or projector works.

```

public abstract class Remote{ 10 usages 2 inheritors 2 zhannurr-png
    protected String id; 1 usage
    protected int volume; 3 usages
    protected Device device; 4 usages

    public Remote(String id, int volume, Device device){ 8 usages 2 zhannurr-png
        this.id = id;
        this.volume = volume;
        this.device = device;
    }

    public abstract String execute(); 8 usages 2 implementations 2 zhannurr-png
}

```

Benefit: Each class has its own responsibility, so the code is easier to understand and change.

2.2 Meaningful names

The variable names clearly show what objects they represent. For example, basicTV means a basic remote connected to a TV, and projector means a projector device.

```

Device tv = new TvDevice();
Device radio = new RadioDevice();
Device projector = new ProjectorDevice();

Remote basicTV = new BasicRemote( id: "A1", volume: 30, tv);
Remote basicRadio = new BasicRemote( id: "A1", volume: 30, radio);
Remote basicProjector = new BasicRemote( id: "A1", volume: 30, projector);

```

Benefit: Meaningful names make the code easier to read and understand without extra comments.

2.3 Small focused methods

The check() method has one simple purpose. It compares the actual result with the expected result and prints whether the test passed or failed.

```

static void check(String id, String classes, String actual, String expected) {
    if (actual.equals(expected)) {
        System.out.println(id + " PASS | " + classes + " | result=" + actual);
        passed++;
    } else {
        System.out.println(id + " FAIL | " + classes + " | result=" + actual);
        System.out.println("expected=" + expected);
    }
}
}

```

Benefit: A small method is easier to read, test, and reuse.

2.4 Avoidance of duplication

All devices use the same Device interface and the same method applySettings(). The remote controls do not need separate code for every device type.

```
public interface Device { 11 usages 2
    String applySettings(int volume);
}

public class TvDevice implements Device { 1 usage
    @Override 2 usages 2 zhannurr-png
    public String applySettings(int volume){
        return "TV | power=ON | volume=" + volume;
    }
}

public class RadioDevice implements Device{ 1 usage 2
    @Override 2 usages 2 zhannurr-png
    public String applySettings(int volume){
        return "Radio | power=ON | volume=" + volume;
    }
}
```

Benefit: We do not need to duplicate the remote-control logic for TV, radio, and projector. New devices can follow the same interface.

2.5 Encapsulation of state

The remote keeps its current device inside the device field.

The setImplementation() method is used to change the device instead of changing it directly from the client code.

```
public void setImplementation(Device device){
    this.device = device;
}

Remote original = basicTV;
String before = original.execute();
original.setImplementation(radio);

Remote afterSwitch = original;
String after = afterSwitch.execute();
```

Benefit: The remote controls its internal device reference through a method. This makes changing the implementation safer and keeps the state inside the Remote class.

3.

I added ProjectorDevice as the third implementation of the device interface. It has the same applySettings() method as the other

devices. And then I created ProjectorDevice objects in Main and added T6 and T7 checks to test it with both BasicRemote and QuietRemote.

The main Bridge structure did not change. Remote still contains the Device reference, and BasicRemote and QuietRemote still work with any Device. The existing Tv and Radio classes and the execute() and setImplementation() methods stayed the same. This shows that a new implementation can be added without changing the existing abstraction hierarchy.

One disadvantage is that Bridge can make the project more complicated because we need more classes and interfaces. For a small project, this may feel like extra code. However, it is useful when we expect to add more types of remotes or devices later.

The main difference is their purpose. Bridge is designed from the beginning to separate two independent parts of a system, so they can change independently. In my project, the two parts are remotes and devices. Adapter is mainly used when we already have an existing class or interface and need to make it work with another interface. So Bridge is about separating two dimensions, while Adapter is about making incompatible interfaces work together.

4. Sources

Refactoring.Guru. *Bridge Design Pattern* <https://refactoring.guru/design-patterns/bridge>

Freeman, E., Robson, E. *Head First Design Patterns*. O'Reilly Media.